# AI-Powered Enterprise E-Commerce Platform

# CI/CD Pipeline Architecture Specification

## Document Information

| Item | Description |
|---|---|
| Project Name | AI-Powered Enterprise E-Commerce Platform |
| Document Type | CI/CD Pipeline Architecture |
| Version | 1.0 |
| Prepared By | Project Team |
| Status | Draft |

## Revision History

| Version | Date | Description |
|---|---|---|
| 1.0 | DD/MM/YYYY | Initial Version |

## Table of Contents

---

# 1. Introduction

Modern enterprise systems require automated software delivery mechanisms.

Continuous Integration and Continuous Deployment (CI/CD) practices help:

- improve development speed,
- reduce human errors,
- increase software quality,
- enable rapid releases.

This document defines the CI/CD architecture for the AI-Powered Enterprise E-Commerce Platform.

---

# 2. Purpose

The purpose of this document is to:

- define automated pipelines,
- standardize software delivery,
- improve release quality,
- support DevOps practices.

---

# 3. CI/CD Objectives

---

## CICD-01

Automate builds.

---

## CICD-02

Automate testing.

---

## CICD-03

Automate deployments.

---

## CICD-04

Reduce deployment failures.

---

### CICD-05

Enable rapid iterations.

---

### CICD-06

Improve software quality.

---

## 4. DevOps Principles

The project follows:

- Automation First
- Infrastructure as Code
- Shift Left Testing
- Continuous Feedback
- Continuous Monitoring

---

## 5. High-Level Pipeline Architecture

```
Developer
     ↓
GitHub Repository
     ↓
GitHub Actions
     ↓
Build
     ↓
Tests
     ↓
Security Checks
     ↓
Docker Images
     ↓
Deployment
     ↓
Monitoring
```

---

## 6. Source Control Strategy

## Repository Platform

Technology:

GitHub

## Branching Model

```
main
develop
feature/*
release/*
hotfix/*
```

## Workflow

```
Create Feature Branch
          ↓
Develop Feature
          ↓
Push Changes
          ↓
Pull Request
          ↓
Review
          ↓
Merge
```

# 7. Continuous Integration (CI)

The CI pipeline executes automatically after:

- Push events
- Pull requests

# CI Activities

## Step 1 – Code Checkout

Retrieve source code.

## Step 2 – Dependency Installation

Install:

- Java dependencies
- Python dependencies
- Frontend dependencies

---

## Step 3 – Static Analysis

Perform:

- Linting
- Formatting checks
- Code quality checks

---

## Step 4 – Build

Build:

- Spring Boot Services
- React Frontend
- FastAPI Services

---

## Step 5 – Unit Testing

Run automated tests.

---

## Step 6 – Generate Reports

Generate:

- Test reports
- Coverage reports

---

# 8. Build Pipeline

---

## Backend Build

Technology:

Maven

Commands:

```
mvn clean package
```

---

## Frontend Build

Technology:

Node.js

Commands:

```
npm install
npm run build
```

---

## AI Build

Technology:

Python

Commands:

```
pip install -r requirements.txt
```

---

# 9. Continuous Testing

Testing activities include:

---

## Unit Tests

Tools:

- JUnit
- PyTest
- React Testing Library

---

## Integration Tests

Validate:

- Service communication

- Database connectivity

---

## API Tests

Tools:

- Postman
- Newman

---

## Security Tests

Tools:

- OWASP Dependency Check
- Snyk

---

## Container Tests

Validate Docker images.

---

# 10. Quality Gates

The pipeline should fail if:

---

## Build Failure

Build unsuccessful.

---

## Unit Test Failure

Critical tests fail.

---

## Coverage Failure

Coverage below target.

Target:

```
80%
```

---

## Security Failure

Critical vulnerabilities found.

---

# 11. Code Quality Pipeline

---

## Static Code Analysis

Tools:

- SonarQube

---

## Quality Metrics

Examples:

- Code Smells
- Duplications
- Bugs
- Vulnerabilities

---

## Quality Gate Conditions

Examples:

- No critical vulnerabilities.
- Coverage >80%.
- No major code smells.

---

# 12. Container Pipeline

---

## Docker Build

Build images for:

```
frontend
backend
recommendation-service
fraud-service
forecast-service
```

---

## Container Registry

Possible Platforms:

- Docker Hub
- GitHub Container Registry

---

## Image Tagging Strategy

```
latest
v1.0.0
v1.1.0
```

---

# 13. Continuous Delivery (CD)

After successful CI:

---

## Deploy to Development Environment

Automatic.

---

## Deploy to Staging Environment

Automatic after approval.

---

## Deploy to Production

Manual approval.

---

# Delivery Flow

```
Build
   ↓
Testing
   ↓
Docker Build
   ↓
Staging
   ↓
Approval
   ↓
Production
```

# 14. Deployment Pipeline

# Development Deployment

Technology:

Docker Compose

# Production Deployment

Future:

Kubernetes

# Example Deployment Sequence

```
Pull Images
      ↓
Stop Old Containers
      ↓
Deploy New Containers
      ↓
Health Checks
      ↓
Traffic Routing
```

# 15. Infrastructure Pipeline

Future improvements:

- Terraform
- Infrastructure as Code
- Automated Environment Provisioning

# 16. Security Pipeline

## Dependency Scanning

Tools:

- Snyk
- OWASP Dependency Check

---

## Secret Detection

Validate:

- API Keys
- Passwords
- Tokens

Possible Tools:

- GitLeaks

---

## Container Scanning

Tools:

- Trivy
- Docker Scout

---

## Security Gates

Pipeline must fail if:

- Critical vulnerabilities detected.
- Secrets exposed.

---

# 17. Monitoring Pipeline

Monitor:

- Deployment failures
- Build failures
- Test failures
- Security findings

---

## Metrics

Examples:

- Build Duration
- Deployment Frequency
- Lead Time
- Mean Time To Recovery

---

# 18. Rollback Strategy

Deployment failures should support rollback.

---

## Strategy 1

Redeploy previous version.

---

## Strategy 2

Restore database backup.

---

## Strategy 3

Rollback container images.

---

# Rollback Flow

```
Failure Detected
       ↓
Stop Deployment
       ↓
Restore Previous Version
       ↓
Health Validation
```

---

# 19. Release Strategy

---

## Semantic Versioning

Format:

```
MAJOR.MINOR.PATCH
```

Example:

```
1.0.0
1.1.0
1.1.1
```

---

## Release Types

- Major Releases
- Minor Releases
- Hotfix Releases

---

# 20. Example GitHub Actions Workflow

```yaml
name: CI Pipeline

on:
  push:
    branches:
      - develop
      - main

jobs:
  build:
    runs-on: ubuntu-latest
```

---

# 21. Future Enhancements

Possible future improvements:

- Kubernetes Deployments
- GitOps
- ArgoCD
- Blue-Green Deployment
- Canary Releases
- Chaos Engineering
- Automated Performance Testing

---

# 22. Recommended Directory Structure

```
.github/
│
├── workflows/
│     ├── backend-ci.yml
│     ├── frontend-ci.yml
│     ├── ai-ci.yml
│     ├── docker-build.yml
│     ├── deploy.yml
│
├── ISSUE_TEMPLATE/
├── PULL_REQUEST_TEMPLATE.md
```

# 23. Recommended Pipeline Architecture Diagram

```
Developer
     ↓
GitHub
     ↓
GitHub Actions
     ↓
Testing
     ↓
Quality Gates
     ↓
Docker Images
     ↓
Staging
     ↓
Production
```

# 24. Success Metrics

| Metric | Target |
|---|---|
| Build Success Rate | >95% |
| Deployment Success Rate | >95% |
| Test Coverage | >80% |
| MTTR | <4 Hours |

| Metric | Target |
|---|---|
| Critical Vulnerabilities | 0 |

# 25. Conclusion

The CI/CD architecture enables:

- automated software delivery,
- improved software quality,
- rapid development,
- reliable deployments,
- and modern DevOps practices.

The pipeline supports future cloud-native expansion and enterprise deployment strategies.

# References

[1] Humble, J., & Farley, D. Continuous Delivery.

[2] GitHub Actions Documentation.

[3] Docker Documentation.

[4] SonarQube Documentation.

[5] OWASP Dependency Check Documentation.

[6] Newman, S. Building Microservices.

[7] Kubernetes Documentation.

[8] DORA DevOps Research and Assessment Reports.